



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 HIGH PERFORMANCE DATA PROCESSING

SECTION 01

**PROJECT 2: REAL TIME SENTIMENT ANALYSIS USING APACHE SPARK AND
KAFKA**

LECTURER: PROF. MADYA. TS. DR. MOHD SHAHIZAN BIN OTHMAN

Group Triple A

Student Name	Matric No.
LAM YOKE YU	A23CS0233
NABIL AFLAH BOO BINTI MOHD YOSUF BOO YONG CHONG	A23CS0252
ANIS SAFIYYA BINTI JANAI	A23CS0049

Submission Date: 26 June 2026

Table of Contents

1. Introduction.....	3
1.1 Background.....	3
1.2 Objectives.....	3
1.3 Project Scope.....	3
1.4 Roles of group members.....	4
2. Data Acquisition and Preprocessing.....	4
2.1 Data Source.....	4
2.2 Tools and Technologies.....	5
2.3 Preprocessing.....	6
3. Sentiment Model Development.....	11
3.1 Model choice.....	11
3.2 Training process.....	11
3.3 Evaluation.....	13
4. Sentiment Analysis Pipelines.....	15
4.1 Streaming Architecture.....	16
4.2 Batch Architecture.....	17
5. Analysis and Results.....	18
5.1 Key findings.....	18
5.2 Visualisation.....	19
5.3 Insights.....	20
6. Optimisation and Comparison.....	20
6.1 Translation Overhead.....	20
6.2 Alternative Sentiment Models.....	22
6.3 Streaming Optimisations.....	23
6.4 Kafka Scalability.....	25
7. Conclusion and Future Work.....	25
8. References.....	26
9. Appendix.....	26

1. Introduction

In this section, the project background and scope were discussed to understand the project and get the full overview of the project.

1.1 Background

In recent years, the rapid development of social media sites such as YouTube has produced a large amount of user-generated content, including comments and messages. Especially in the fields of restaurant evaluation, product feedback, and service rating, these text data contain rich public opinions and can be used to analyze their sentiment trends.

However, due to the large number and loose structure of these comments, manual analysis is difficult to achieve. Therefore, with the help of scalable data processing frameworks such as Apache Spark and Apache Kafka, real-time sentiment analysis of streaming data can be carried out.

This project aims to use machine learning and distributed data processing technology to build a real-time emotional analysis system for Malaysian food reviews on YouTube. In order to compare the performance and show the scalability of contemporary big data systems, the system combines batch processing and stream processing.

1.2 Objectives

The objectives of this project are as follows:

- 1) To gather and preprocess YouTube comments related to Malaysian food.
- 2) To classify the data set by building a labelled sentiment dataset using automatic labelling.
- 3) To develop and obtain the best sentiment classification model by comparing several models such as Naive Bayes and LSTM.
- 4) To develop real-time processing pipelines using Apache Spark and Apache Kafka.
- 5) To evaluate system performance between batch and streaming architecture.
- 6) To visualise sentiment insights using Elasticsearch and Kibana.

1.3 Project Scope

This project focuses on:

- 1) Food-related comments on the Youtube platform from selected videos.
- 2) 3 sentiment classes consist of positive, neutral, and negative.
- 3) Batch and real-time processing pipelines.

- 4) Model deployment using a trained LSTM classifier.
- 5) Performance comparison on the processing speed and scalability.

1.4 Roles of group members

The following table shows the roles and primary responsibilities of each member.

Table 1. Roles and responsibilities of group members

Roles	Primary Responsibilities	Member
Data Engineer	• Manages data acquisition • Preprocessing	Anis Safiyya
NLP and Model Engineer	• Develops, trains, and evaluates the sentiment models • Prepares the model comparison.	Nabil Aflah Boo
Group Leader and Pipeline Engineer	• Builds the Kafka and Spark pipeline • Configures storage • Develops the dashboards.	Lam Yoke Yu

2. Data Acquisition and Preprocessing

This section details the overall process of data collection and preprocessing, which is a key step in converting the original data of YouTube comments into a structured and clean dataset suitable for sentiment analysis. It includes methods such as data collection, cleaning, filtering, and conversion, which will be carried out before model training and evaluation. The goal of this stage is to ensure the quality, consistency, and relevance of the data to the project.

2.1 Data Source

The dataset used in this project is extracted from YouTube comments using the YouTube data API. This API allows access to the public comment area of the video. The data collected are related to Malaysian food videos, such as restaurant reviews, street food experiences, and food vlogs. These videos are selected to show the diverse food culture and international food discussion.

Source for Scrapping:

1. <https://www.youtube.com/watch?v=rakfl1KCaMk>
2. <https://www.youtube.com/watch?v=tG8ZaAgCTiE>
3. <https://www.youtube.com/watch?v=etTe2i4GkWo>
4. <https://www.youtube.com/watch?v=nqPoocCKci8>

5. https://www.youtube.com/watch?v=_PfAYdYN_D0
6. <https://www.youtube.com/watch?v=5Bd0wjyJJY&t=824s>
7. <https://www.youtube.com/watch?v=xt4SsI6xcss>
8. <https://www.youtube.com/watch?v=CsRezdfnqcg>
9. https://www.youtube.com/watch?v=0BScF_gdj8g
10. <https://www.youtube.com/watch?v=4Fy4jMU9JqI>
11. <https://www.youtube.com/watch?v=6-gyPec-NSA>
12. <https://www.youtube.com/watch?v=8a5bA1HMPD0>
13. <https://www.youtube.com/watch?v=KIsScVxMHBQ>
14. <https://www.youtube.com/watch?v=6I4g9qI72o0>
15. <https://www.youtube.com/watch?v=K7DeqmLxVFM>
16. https://www.youtube.com/watch?v=_lZjm3mMl3g
17. https://www.youtube.com/watch?v=qYZ22_54N9I
18. <https://www.youtube.com/watch?v=iVsn8X5K7pw>
19. <https://www.youtube.com/watch?v=feDph2WgRfI>
20. <https://www.youtube.com/watch?v=YyaXll-a6hs>
21. <https://www.youtube.com/watch?v=s4uwJ-k3m0o>
22. <https://www.youtube.com/watch?v=bF1VamLDso4>
23. <https://www.youtube.com/watch?v=DBs3YyNh8xg>
24. <https://www.youtube.com/watch?v=z73Ct-4pueo>

A total of 37,932 comments have been collected from the 24 selected videos. These comments include opinions on food and other topics, junk messages, and dialogues. Before it is used for model training and analysis, this dataset needs preprocessing including data cleaning, filtering and normalization. This is to ensure the accuracy and reliability of the data collected.

2.2 Tools and Technologies

The following tools were used in this project:

- YouTube Data API (data collection)
- Python (data processing)
- Pandas (data handling)
- langdetect (language detection)
- Google Translator API (translation)
- Apache Spark (stream processing)
- Apache Kafka (message streaming)

2.3 Preprocessing

The preprocessing pipeline was designed to improve data quality and construct a reliable sentiment-labelled dataset from the scraped comments. The overall workflow is illustrated in Figure 1.



Figure 1. Preprocessing flow

Text Cleaning

Raw comments collected from online sources often contain noise such as HTML tags, URLs, user mentions, hashtags and irregular formatting. These elements increase vocabulary variation and may negatively affect model performance. Therefore, text cleaning was performed to standardise comment content while preserving the semantic information required for sentiment analysis.

The cleaning process included removing HTML tags and URLs, converting text to lower case, replacing user mentions with a common placeholder, removing hashtags and HTML entities, normalising repeated characters and removing excessive whitespace.

32

Remove unused row

Following text cleaning, comments that were unlikely to provide meaningful information were removed. This included null or empty comments, duplicate entries, comments consisting primarily of numbers or symbols, spam-like comments, and comments that were excessively short or excessively long. After data removal, the total number of data was reduced from 37,932 records to 32,368 records.

This filtering step reduced noise within the dataset and ensured that only comments containing sufficient textual information were retained for subsequent analysis.

Language Detection and Translation

The collected comments contained multiple languages due to the diverse backgrounds of online users. Therefore, all comments were standardised into English before sentiment labelling.

First, language detection was performed using the langdetect library to identify the primary language of each comment. Comments already written in English were retained without modification. A total of 9226 comments were flagged as non-english comments. Comments identified as non-English were automatically translated into English using Google Translator.

This process ensured that all comments were represented in a consistent language, allowing the sentiment analysis models to process the dataset uniformly and reducing potential performance variations caused by multilingual input.

Sentiment Labelling

Sentiment labels were generated automatically using an ensemble of three pre-trained transformer-based sentiment analysis models:

- Twitter-RoBERTa (cardiffnlp/twitter-roberta-base-sentiment-latest)
- XLM-RoBERTa (cardiffnlp/twitter-xlm-roberta-base-sentiment)
- Multilingual Sentiment Analysis (tabularisai/multilingual-sentiment-analysis)

Each model independently predicted a sentiment label and an associated confidence score for every comment. Since the models used slightly different label schemes, labels were first standardised into three sentiment classes: positive, neutral, and negative.

Then, a majority voting strategy was applied. The final sentiment label was determined according to the agreement among the three models:

- Full agreement: all three models predicted the same sentiment label
- Majority agreement: two models predicted the same sentiment label
- No agreement: all three models predicted different sentiment labels

For comments with full agreement, the final confidence score was calculated as the average confidence score of all three models. For comments with majority agreement, the confidence score was calculated as the average confidence score of the two agreeing models. Comments with no agreement were marked for review and were not automatically assigned a sentiment label. A simple example of the voting mechanism is depicted in Table 2.

Table 2. Voting mechanism example

roberta_label	xlmr_label	multi_label	final_label	agreement	confidence
positive	positive	positive	positive	full	Avg(all 3 scores)
positive	positive	neutral	positive	majority	Avg(agreeing scores)
positive	neutral	negative	review	none	0

The result of the labelling process is shown in Figure 2.

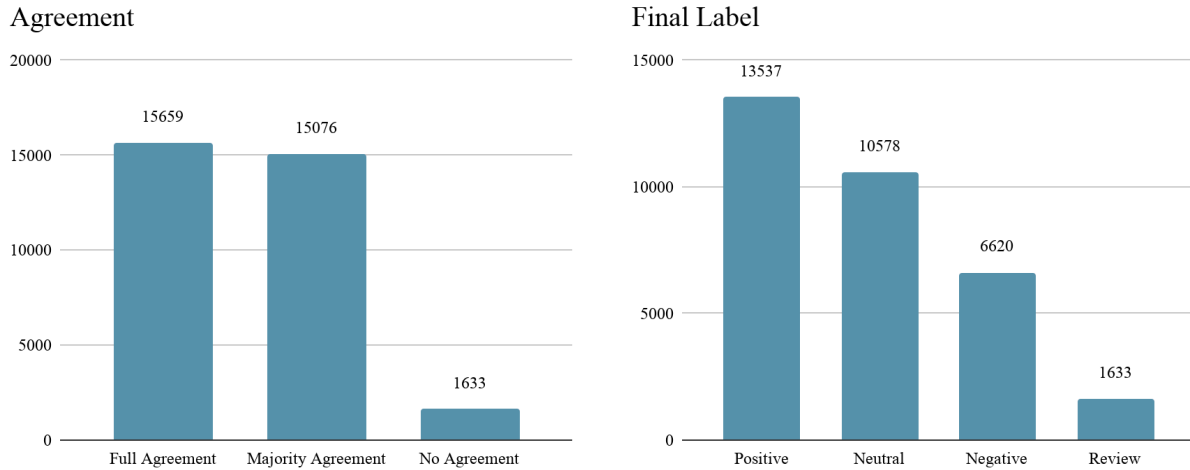


Figure 2. Agreement and final label distribution

Based on Figure 2, there are 1633 comments that did not reach agreement among the three models. Positive comments constitute the majority of the dataset, while negative comments represent the smallest proportion.

This ensemble approach reduces the influence of errors from individual models and produces more reliable sentiment annotations than relying on a single model.

Remove Low Confidence Label

Although majority voting improves labelling consistency, some comments may still receive uncertain sentiment predictions. To improve the quality of the final dataset, only comments with sufficiently reliable labels were retained.

A comment was accepted only if:

- All three models agreed on the sentiment label (full agreement), or
- Two models agreed on the sentiment label and the average confidence score of the agreeing models was at least 0.60.

Although comments with no agreement were initially marked for review, it was decided to just drop them, leaving a total of 25315, with the distribution shown in the following figure.

Dataset Distribution with High Confidence Label

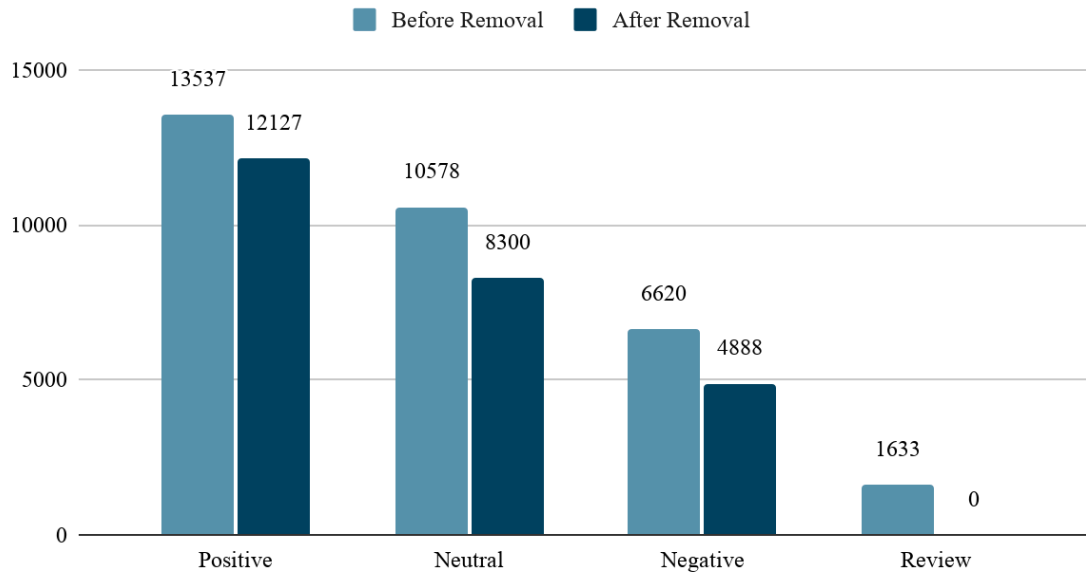


Figure 3. Dataset Distribution with High Confidence Label

Filter Food Related Comments

Although the comments were sourced from YouTube videos about food, many comments discussed unrelated topics such as content creators, filming quality, travel experiences or general conversations. Therefore, an additional filtering stage was required to retain only comments directly related to food.

A keyword-based filtering approach was adopted. A predefined list of food-related keywords was constructed based on common food terms, restaurant-related vocabulary, ingredients, dishes, beverages, and food items in both English and Malay. Examples include terms such as food, restaurant, meal, burger, pizza, curry, rice, noodle, ayam, ikan and durian.

Each comment was converted to lower case and checked for the presence of at least one keyword from the predefined list. Comments containing one or more food-related keywords were marked true, while comments without any matching keywords were marked false.

There are a total of 13092 (51.7%) comments labelled as food related and 12223 comments (48.3%) comments marked as non-food related.

This filtering step ensured that the final dataset remained aligned with the study domain and reduced the influence of comments discussing unrelated topics.

Manual Review

The keyword-based filtering approach is unreliable, therefore a manual review was conducted to identify misclassified and irrelevant comments. This process helped improve the overall reliability of the dataset.

Table 3. Keyword-based filter and manual review comparison

		Manual Review	
		False	True
Keyword-based filter	False	11757	466
	True	4055	9037

Therefore, only 9037 relevant records were preserved. The final proposition of sentiments are shown in the following figure.

Sentiment Distribution

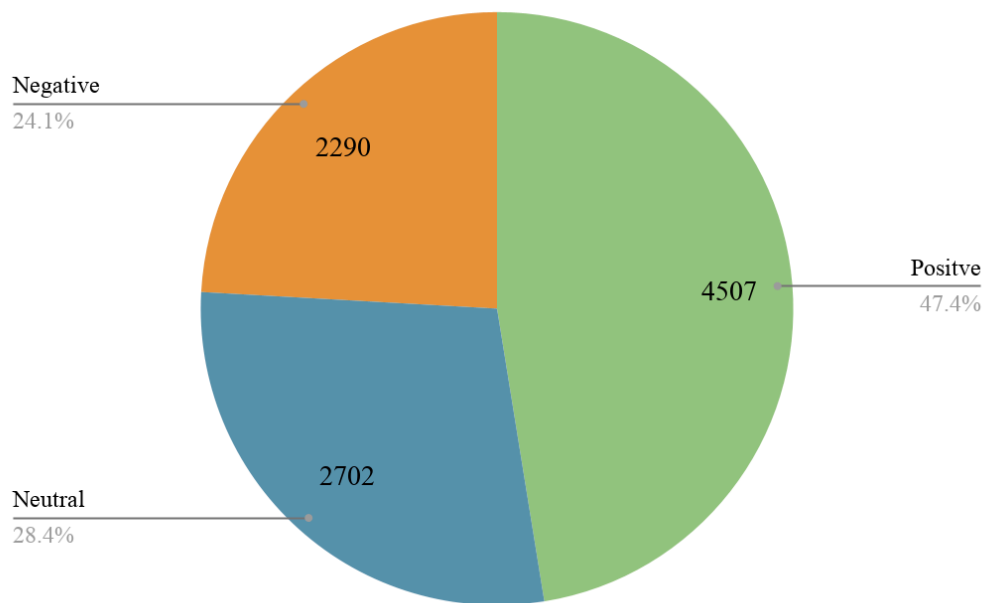


Figure 4. Sentiment Distribution

The final dataset remains positively skewed, with positive comments accounting for 47.4% of all records. Neutral and negative comments account for 28.4% and 24.1% of the dataset respectively.

In summary, the preprocessing pipeline reduced the dataset from 37,932 scraped comments to 9,037 food-related comments with reliable sentiment labels. The combination of text cleaning, language standardisation, ensemble sentiment labelling, confidence-based filtering, domain specific filtering, and manual review helped ensure that the final dataset was both relevant to the study objectives and suitable for subsequent model training and evaluation.

3. Sentiment Model Development

In this section, model selection rationale, training process, and evaluation approach that are used in building the sentiment classifier for Malaysian Food Reviews will be documented.

3.1 Model choice

There are two models that have been selected and trained in parallel for a 3-class sentiment classification task: negative, neutral, and positive. The models that are chosen are:

Model 1: Multinomial Naive Bayes

This model is a typical probabilistic classifier that has been simple, fast, and effective in the text classification tasks. It combined with TF-IDF vectorization front-end that makes it interpretable and lightweight, well suited for deployment in streaming environments such as Apache Spark.

Model 2: Long Short-Term Memory (LSTM)

This model is a recurrent neural network architecture, which is able to learn sequential dependencies and maintain context across the text. It is more expressive for subtle sentiment signals as it employs a learned embedding layer instead of hand-crafted features.

These models will be tested and the best model with highest F1-Score will be chosen as the production model.

3.2 Training process

All of the text basically goes through a common pipeline before starting the model specific steps which includes lowercasing, stopword removal using Natural Language Toolkit (NLTK) English stopwords, porter stemming, and training, validation, and test split for reproducibility.

Naive Bayes Training

At the beginning, a TF-IDF vectorization has been done. The cleaned text is then turned into numeric features by using `TfidfVectorizer`, with `max_features` set to 10,000, `ngram_range` of (1,2) so it can catch unigrams together with bigrams, and `min_df=2` so the rarer tokens that only showed up less than two times get dropped. After that, the model fit part where a Multinomial Naive Bayes model with `alpha=0.1` was fitted directly on top of the TF-IDF training matrix.

LSTM Training

The first step of LSTM training is hyperparameters where the maximum length of the sequence is calculated dynamically based on 90th percentile of lengths of training text with a maximum of 15 tokens per sequence. The model is trained with a vocabulary size of 8,000 words using MAX_WORDS, an embedding dimension of 32, an LSTM unit size of 64, a batch size of 32, and a maximum of 30 epochs of training with early stopping.

Second, it goes through a class balancing where the model random oversampling with replacement is used to overcome the class imbalance in the training set by resampling each class to the size of the largest class. After that, the class weights are worked out with `sklearn.utils.class_weight.compute_class_weight` and then fed into training so that the minority classes keep getting harsher penalties when they're misclassified.

Then, the wording is tokenized by a Keras Tokenizer that is fitted on the training corpus, and there is also a <OOV> token used. Every resulting sequence is then padded out to MAX_LEN with pre-padding so that all sequences come out to the same length, even across different batches.

Fourth, in architecture, the network consists of 6 layers which play different roles in learning the representations of sentiment from the text:

Table 4. LSTM Network Architecture

Layer	Details
Embedding	MAX_WORDS x EMBED_DIM, input length MAX_LEN
SpatialDropout1D	Rate 0.2
LSTM	64 units, dropout 0.3, recurrent dropout 0.2
Dense	32 units, ReLU activation
Dropout	Rate 0.4
Dense (output)	3 units, Softmax activation

From Table 4, the input first is fed into an embedding layer which converts each token index to a dense 32 dimensional vector, thus the model is able to learn semantic word representations during training. The second layer, SpatialDropout1D with a rate of 0.2 is added to address overfitting in sequential data by dropping feature maps instead of the elements. A sequence of 64 units is then passed to an LSTM layer where it extracts temporal dependencies between tokens, followed by dropout applied to the inputs of 0.3, and recurrent dropout to hidden states of 0.2 to further regularize the recurrent connections. Next, the output fed into a dense layer of 32 units

with ReLU activation to learn non-linear combinations of the features extracted by the LSTM, and a dropout layer with rate 0.4 to prevent co-adaption of neurons before making the final classification. The output, which is a dense layer, gives a 3 unit Softmax distribution, that somehow maps the Negative, Neutral, and Positive sentiment classes. The model used categorical_crossentropy as a loss function, and Adam as the optimizer, with a learning rate set to 5e-4.

The last step of LSTM is the callbacks function where there are two callbacks controlling the dynamics of training. EarlyStopping tracks val_accuracy with patience of 6 epochs, stopped training process and saved the best weights. The ReduceLROnPlateau keeps track of val_loss and decreases the learning rate by a factor of 0.5 and 3 consecutive epochs with no significant decrease in the learning rate.

3.3 Evaluation

The two models are tested on the held-out test set, 20% of the complete dataset. Accuracy, precision, recall, and F1 score are computed using weighted averaging to balance out any residual class imbalance, for performance measurement purposes. The report for each class classification is created for each model with detailed analysis of performance in negative, neutral, and positive categories.

Table 5. Model Comparison Table Between Naive Bayes and LSTM

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	0.6911	0.6865	0.6911	0.6833
LSTM	0.7311	0.7283	0.7311	0.7276

From Table 5, the LSTM model archives an accuracy of 0.7311, a precision of 0.7283, a recall of 0.7311, and a weighted F1 score of 0.7276, which are better than Naive Bayes model with 0.6911, 0.6865, 0.6911, and 0.6833 recorded respectively. This shows that the LSTM is able to create a meaningful contribution to the bag-of-words model for predicting the context of word in a sequence.

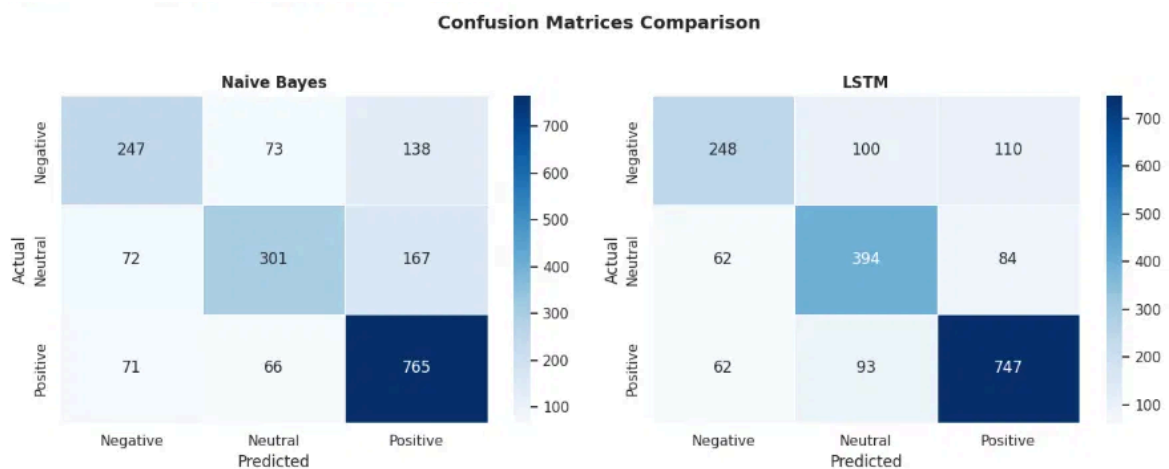


Figure 5. *Confusion Matrices Comparison Between Naive Bayes and LSTM*

From Figure 5, the positive class has the best prediction by both of the models, with Naive Bayes correctly classifying 765, and LSTM for 747 cases. The LSTM seems to do really well on the Neutral class, it correctly classifies 394 out of 395 cases, while Naive Bayes only manages 301 correct cases. This highlights the fact that the language used for neutral sentiment is more subtle and hence is better captured by a sequential modelling approach. Both models have the highest number of misclassifications at the boundary between negative and positive class for both models, as mixed-tone reviews often occur in the sentiment analysis.

Best Model Selection

In terms of production, the model with the top weighted F1 score is automatically picked. As LSTM is showing a higher F1 score of 0.7276, it gets saved as `best_model.keras`. Then all the artefacts such as fitted vectorizer, tokenizer, configuration files, full predictions, and the metric comparison table, are stored under `./saved_models/` in a ready to use format, and they are also able to integrate with Apache Spark Structured Streaming.

4. Sentiment Analysis Pipelines

Figure 6 illustrates the overall system architecture developed for the sentiment analysis system. The architecture consists of two main components, a model development workflow and two sentiment analysis pipelines.

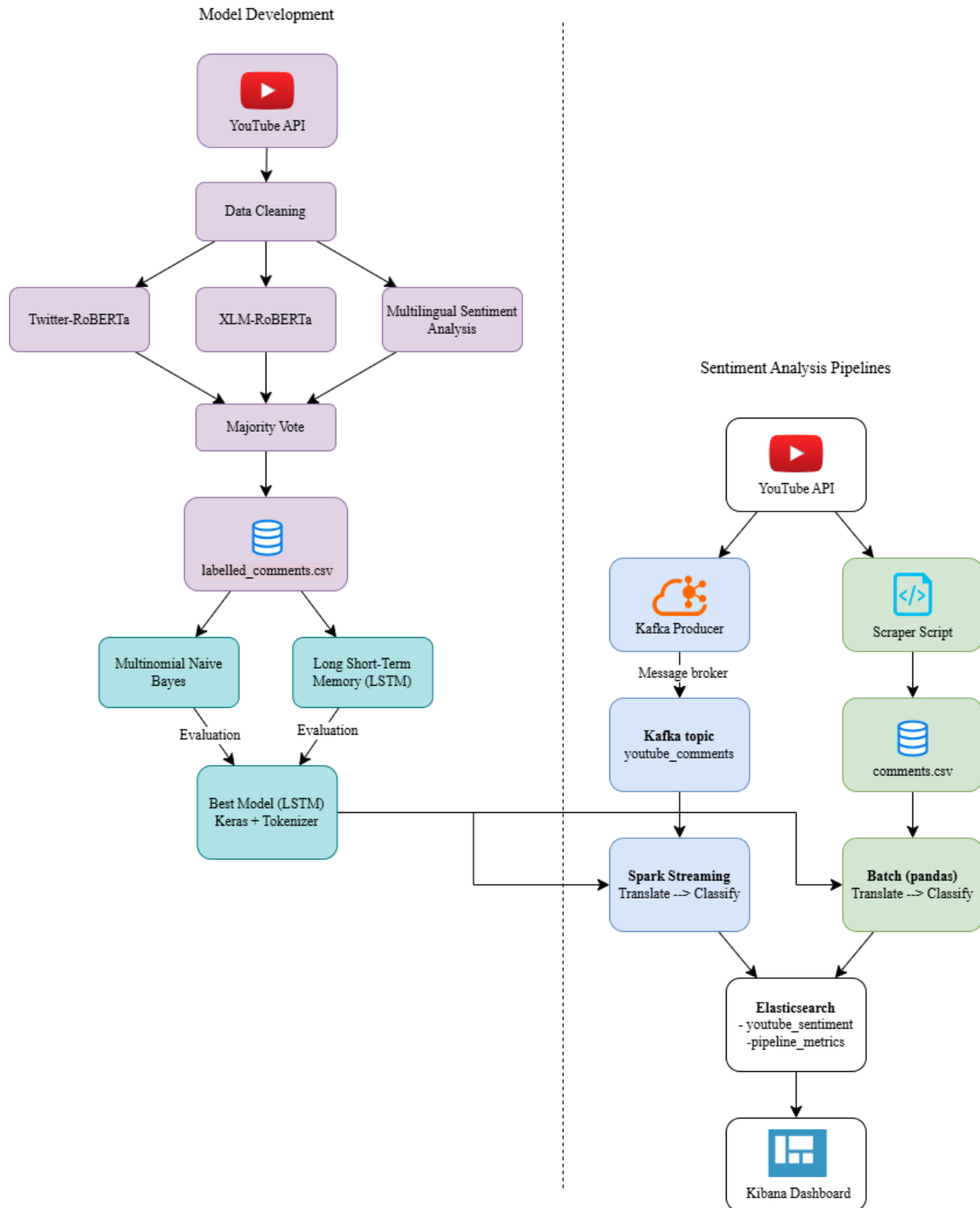


Figure 6. System Architecture

The model development workflow begins with collecting YouTube comments, followed by data preprocessing and automatic sentiment labelling using three pre-trained transformer-based models. A majority voting strategy is applied to determine reliable sentiment labels and the resulting high-confidence dataset is used for model training. The Multinomial Naive Bayes and LSTM models are evaluated. The best-performing model, the LSTM model, is selected and integrated into the processing pipelines.

The streaming pipeline is designed to continuously collect newly posted YouTube comments, process incoming data through Apache Kafka and Apache Spark Structured Streaming, perform sentiment prediction using the trained LSTM model and store the prediction results in Elasticsearch for visualisation and analysis.

In contrast, the batch pipeline processes a collection of comments stored in a CSV file. The collected comments are processed in a single batch without the use of Kafka and Spark, allowing the processing performance to be compared with the streaming approach.

4.1 Streaming Architecture

The streaming architecture was developed to perform near real-time sentiment analysis on YouTube comments. The overall workflow is illustrated in Figure 7.



Figure 7. Streaming Architecture

YouTube Data Producer (stream_producer.py)

The process begins with a YouTube data producer, which continuously polls the YouTube Data API every 60 seconds to retrieve newly posted comments from selected videos. To prevent duplicate processing, a timestamp watermark mechanism is implemented to ensure that only comments published after the previous polling cycle are collected. The retrieved comments are then published to the youtube_comments Kafka topic.

Apache Kafka

Apache Kafka acts as the message broker within the architecture. By decoupling data ingestion from data processing, Kafka provides a buffer between producers and consumers, allowing comments to be ingested and processed independently. This improves system scalability and reliability.

Apache Spark Structured Streaming (stream_spark.py)

Apache Spark Structured Streaming serves as the stream processing engine. Spark continuously consumes comment data from the Kafka topic and processes the incoming records in micro-batches. For each batch, the JSON messages are parsed, comments are translated into English, and the trained LSTM sentiment classification model is loaded to generate sentiment predictions. The model outputs both a sentiment label and an associated confidence score for each comment.

Elasticsearch

The processed results are subsequently stored in Elasticsearch. Two indices are maintained within Elasticsearch. The `youtube_sentiment` index stores the sentiment analysis results, including the original comment, translated comment, sentiment label, confidence score and associated metadata. The `pipeline_metrics` index stores performance metrics such as batch size, processing duration and throughput, which are used to evaluate system performance.

Kibana

Finally, Kibana is used as the visualisation layer. By connecting directly to Elasticsearch, Kibana enables real-time monitoring of sentiment distributions, comment activity, and pipeline performance metrics.

4.2 Batch Architecture

To provide a baseline for comparison, a batch-processing architecture was also implemented. Unlike the streaming architecture, the batch approach does not utilise Apache Kafka or Apache Spark. Instead, comments are collected, stored, and processed periodically as a single dataset.

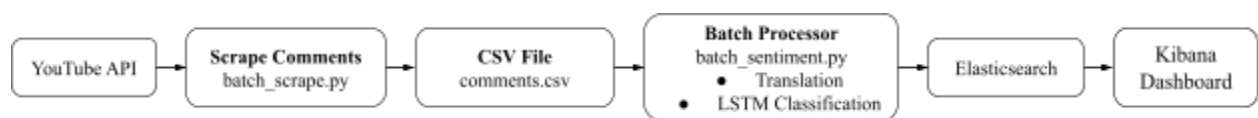


Figure 8. Batch Architecture

YouTube Data Collector (batch_scrape.py)

The process begins with the batch data collector, which retrieves comments from the YouTube Data API and stores them in a CSV file.

Batch Processing (batch_sentiment.py)

Once data collection is complete, the batch processing script reads the entire dataset and performs sentiment analysis on all comments in a single execution.

Similar to the streaming architecture, comments are translated into English when necessary before being passed to the trained LSTM model for sentiment prediction. The generated sentiment labels and confidence scores are then stored in Elasticsearch together with the corresponding comment metadata.

Although both architectures utilise the same sentiment analysis model and storage layer, they differ significantly in their processing strategy. The batch architecture processes all collected comments at once after data collection has completed, whereas the streaming architecture continuously processes comments as they arrive.

5. Analysis and Results

This section introduces the analysis results and findings of the sentiment analysis system developed in the project. The results come from the batch and streaming output of Elasticsearch and are displayed in Kibana. It was analyzed based on emotional distribution pattern, model confidence behavior, comparison of batch processing and streaming output, and time trends.

5.1 Key findings

A key finding of sentiment distribution analysis is that users' emotions are seriously biased towards positive. The overall distribution of comments is also obviously biased towards positive comments, with neutral comments accounting for the majority and negative comments accounting for the smallest proportion. This shows that the Malaysian public generally has a positive attitude towards food-related content on YouTube. The positive aspects related to food culture on social media, such as food content, can stimulate users' appreciation, satisfaction, and positive experience.

In addition, we also analyzed the overall sentiment distribution and compared batch processing and streaming processes to gain an in-depth understanding of system behavior. The batch results show that the system is stable, because all data is processed in one batch. However, due to the lack of a real-time processing mechanism, its speed is relatively slow. The streaming process is the opposite, as all incoming comments are processed in the form of streaming and provided to other systems with low latency and high response speed. This makes the streaming method more suitable for real-time sentiment monitoring, especially for scenarios that require real-time sentiment feedback. The two architectures are also compared in terms of stability and response speed during processing.

Another important conclusion can be drawn from the distribution of confidence scores: between 2016 and 2017, the confidence in the sentiments decreased significantly. Most of the prediction results of the model are in the medium to high confidence range, usually 0.6 to 0.9. This shows that the confidence of the trained LSTM model for sentiment classification of clear emotional expression is generally high. When comments contain ambiguous language, mixed emotions, or multilingual comments, the credibility will be slightly reduced. This phenomenon shows that although the model performs well with structured emotional patterns, it is still sensitive to noisy text and mixed language text common in YouTube comments.

The emotional distribution analysis of different videos also shows significant differences in user behavior. The emotional characteristics of videos vary. Some videos get more positive feedback than others, while some videos receive more neutral or negative feedback. These comparisons are affected by many factors, including the quality of the video content, the presentation method, and the expectations of the audience. In some cases, comments may even deviate from food-related discussions, thus affecting the interpretation of overall results. This shows that emotional tendencies are inconsistent in the whole data set and are closely related to the context of the specific video.

Finally, the timing trend analysis based on time stamp data shows that there is no significant trend change in emotional distribution during data collection. There are no obvious peaks in the sentiment analysis, which means that user behavior is consistent in time. The streaming system can continuously capture incoming comments without interrupting the data flow.

5.2 Visualisation

The visual chart generated by Kibana shows the sentiment behavior of the data set in detail. Multiple dashboards were designed for this analysis, such as sentiment distribution diagrams, comparative views of batch processing and streaming processing, confidence score distribution analysis, and video-level sentiment classification.

These dashboards allow users to explore data interactively in various ways as users can filter data according to processing patterns, sentiment label, video identifiers and timestamp to discover patterns and compare system performance in batch and streaming modes. The dashboard can present the output log more clearly because they can extract meaningful information from structured data.



Figure 9. Kibana Visualization Dashboard

5.3 Insights

Based on the overall analysis, we can draw some important conclusions about the system. First, the high positive sentiment shows that Malaysian YouTube users are generally more interested in Malaysian cuisine, and their positive experience also confirms this, which shows that they have a strong interest in food-related content.

Secondly, the analysis of batch processing and streaming architecture shows that streaming architecture is more suitable for real-time emotional monitoring. Batch processing is suitable for historical analysis and offline scenarios, but not for handling dynamic data flows that require quick response.

Third, the confidence distribution shows that the LSTM model can effectively classify clear emotional expressions, but it is not effective for ambiguous or multilingual comments. This shows that the classification effect can be further improved by using multilingual models or better handling mixed-language texts.

Fourth, the emotional distribution of different videos shows the sentiment of the context. The sentiments of the comments are affected by many factors, including the type of video, the style of the creator, and the expectations of the audience.

In addition, the successful integration of Kafka, Spark, Elasticsearch, and Kibana proves that the system is scalable and can be used in large-scale emotional analysis applications. The system is carefully designed to process batch data and streaming data efficiently and stably.

6. Optimisation and Comparison

There are several optimisations that could be implemented to the current pipeline.

6.1 Translation Overhead

The current implementation in both the streaming and batch processing pipelines translates comments into English before sentiment prediction. The use of Google Translate introduces additional processing overhead because every comment requires an external API request before prediction can be performed.

To investigate the impact of translation on system performance, two experiments were conducted using the same set of 60 comments. The first experiment translated every comment directly before prediction. The second experiment introduced a language detection stage, where only non-English comments were translated.

A modified version of the batch processing script was used to measure the average processing time of each stage. The output snippets of both experiments are shown in Figure 10 and Figure 11.

Translation Time	Prediction Time
0.4610s	0.4011s
0.4742s	0.0935s
0.4173s	0.0970s
0.4310s	0.0954s
0.4490s	0.0756s
0.5134s	0.0805s
0.3986s	0.0756s
0.4784s	0.0625s
0.4126s	0.0845s
0.4552s	0.0813s
0.4656s	0.0812s
0.4690s	0.0840s
0.3810s	0.0634s
0.4703s	0.0579s
0.3789s	0.0570s

Figure 10. Snippet of Translation Time and Prediction Time

Detected Language	Detection Time	Translation Time	Prediction Time
unknown	0.3462s	0.4103s	0.3956s
so	0.0070s	0.3872s	0.0632s
en	0.0040s	0.0000s	0.0769s
fa	0.0022s	0.4026s	0.0772s
en	0.0020s	0.0000s	0.0656s
en	0.0080s	0.0000s	0.0694s
en	0.0010s	0.0000s	0.0900s
fa	0.0010s	0.4185s	0.1071s
fa	0.0036s	0.5044s	0.0802s
fa	0.0010s	0.4405s	0.0852s
ur	0.0061s	0.4016s	0.1086s
fa	0.0020s	0.4860s	0.0810s
fa	0.0035s	0.5053s	0.1329s
fa	0.0028s	0.4593s	0.0740s

Figure 11. Snippet of Language Detection Time, Translation Time and Prediction Time

The average processing times obtained from both experiments are summarised in Figure 11.

Processing Time for each Procedures

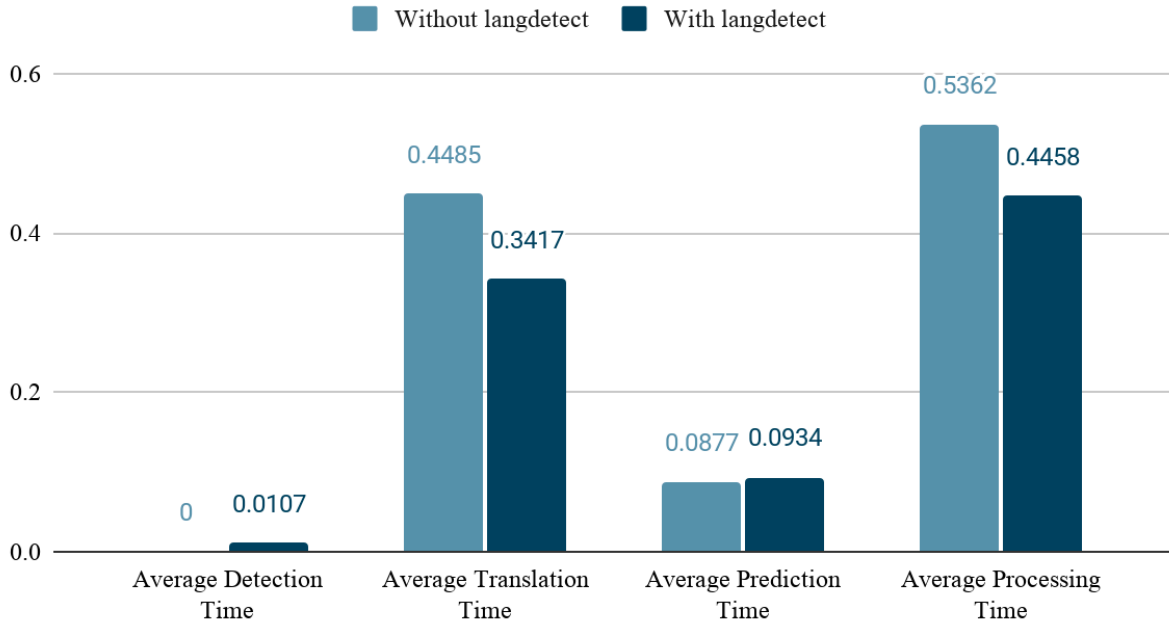


Figure 12. Processing time comparison with and without langdetect

The results show that translation is the most time-consuming operation in the sentiment analysis pipeline. When every comment is translated, the average translation time is 0.4485 seconds per record, which is substantially higher than the average prediction time of 0.0877 seconds.

After introducing language detection, the average translation time decreased to 0.3417 seconds per record because English comments no longer required translation. Although language detection introduced a small overhead of 0.0107 seconds per record, the reduction in translation requests improved overall throughput from 1.86 records per second to 2.24 records per second.

These findings indicate that translation is one of the bottlenecks in the current architecture. Therefore, incorporating language detection before translation can improve processing efficiency by reducing unnecessary translation operations while introducing only a negligible additional cost.

6.2 Alternative Sentiment Models

The current sentiment analysis pipeline utilises an LSTM model. During model evaluation, the LSTM achieved an accuracy of 0.7311 and an F1-score of 0.7276. Although these results are satisfactory for the objectives of this study, alternative deep learning architectures may potentially improve classification performance or processing efficiency.

Some models that can be considered include Bidirectional LSTM (BiLSTM), Gated Recurrent Units (GRUs), BERT and Multilingual Models.

One possible alternative is the Bidirectional Long Short-Term Memory (BiLSTM) model. Unlike the current unidirectional LSTM, a BiLSTM processes text sequences in both forward and backward directions, allowing contextual information from both preceding and succeeding words to be captured, potentially improving sentiment classification performance. However, it may be more computational intensive.

Another potential alternative is the Gated Recurrent Unit (GRU). GRUs belong to the same family of recurrent neural networks as LSTMs but employ a simpler architecture with fewer trainable parameters. This may reduce training and inference time while maintaining comparable classification performance.

Transformer-based architectures such as BERT may also be considered. BERT has demonstrated strong performance across a wide range of natural language processing tasks due to its ability to capture contextual relationships between words more effectively than traditional recurrent neural networks. However, their computational requirements are substantially higher than those of LSTM-based models.

As discussed in Section 6.1, translation was identified as the primary bottleneck within the sentiment analysis pipeline. Therefore, multilingual transformer models could be considered as an alternative approach. Models capable of directly processing multiple languages would eliminate the translation stage and potentially reduce overall processing latency.

6.3 Streaming Optimisations

The current streaming implementation processes sentiment predictions on the Spark driver node. Within the `foreachBatch` function, the `collect()` operation is used to retrieve all records from the Spark DataFrame and transfer them to the driver before prediction is performed.

Python

Reduced code snippet

```
def process_batch(batch_df, batch_id):
    rows = batch_df.collect()    # Transfer record from Spark to driver node

    for r in rows:                # Prediction loop as normal Python code
        translated = translate_to_english(comment)
        label, confidence = predict(translated)
```

While this approach simplifies implementation and model integration, it limits the benefits of Spark's distributed computing architecture. Since translation and sentiment prediction are executed sequentially on the driver node, processing performance becomes constrained by the resources of a single machine.

A potential optimisation is to distribute the prediction workload across Spark executors using User Defined Functions (UDFs), Pandas UDFs or model-serving approaches. By performing sentiment prediction directly on worker nodes, multiple comments can be processed in parallel, allowing the system to better utilise Spark's distributed processing capabilities and improve scalability for larger workloads. The comparison of the current implementation and suggested optimised implementations is shown in the following figure.

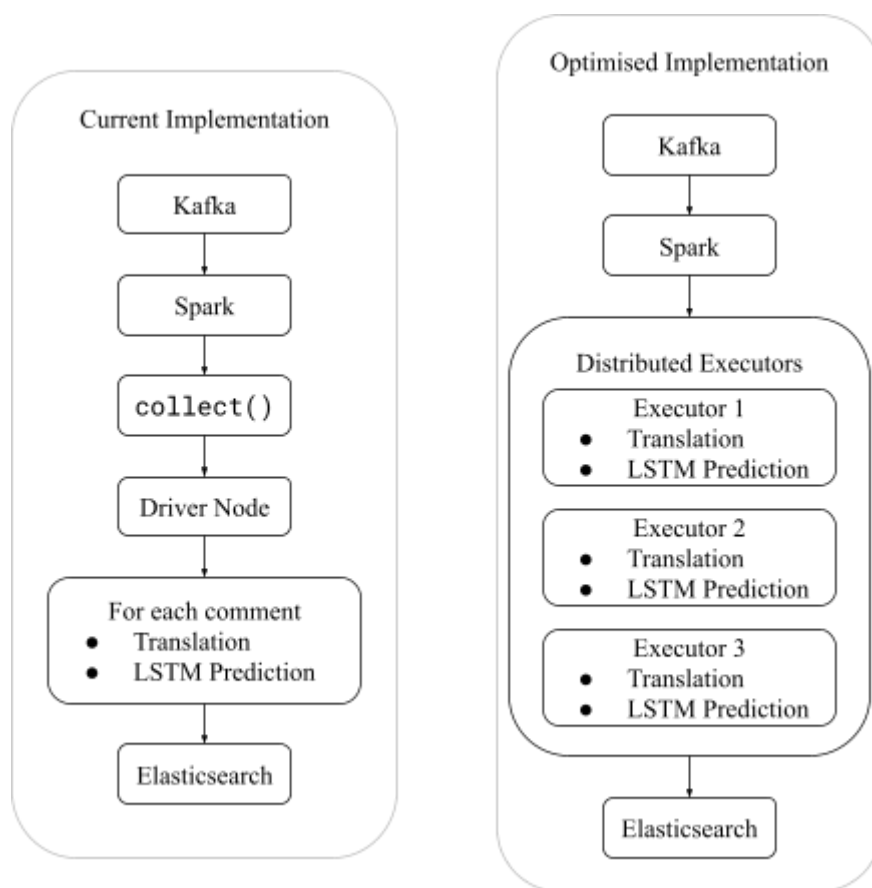


Figure 13. Comparison of current implementation and suggested optimised implementation

Although distributing translation and prediction across Spark executors can improve scalability, translation remains dependent on an external service. Many simultaneous translation requests could hit Google Translate rate limits or introduce network instability. A further optimisation

would be replacing the translation step with a multilingual sentiment model, allowing comments in different languages to be directly classified without translation.

6.4 Kafka Scalability

The current streaming architecture uses a single Kafka topic (youtube_comments) consumed by one Spark Structured Streaming application. While this configuration is sufficient for the current workload, it may limit scalability when processing a larger volume of incoming comments.

Kafka improves scalability through topic partitioning, where a topic can be divided into multiple partitions. Each partition can be consumed independently, allowing multiple consumers to process messages in parallel. Therefore, increasing the number of partitions and deploying multiple consumer instances could improve message processing throughput and reduce processing latency. The comparison of the current implementation and suggested optimised implementations is shown in the following figure.

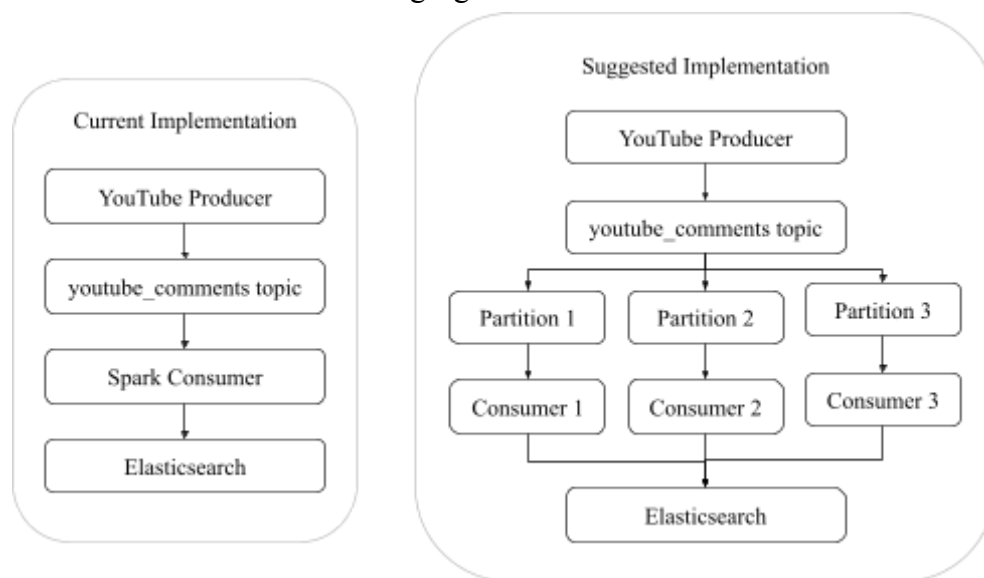


Figure 14. Comparison of current implementation and suggested optimised implementation

With multiple partitions and consumers, incoming comments can be distributed across different processing units. This allows the system to handle higher data ingestion rates and improves fault tolerance, as failure of one consumer does not stop the entire pipeline.

However, increasing partitions also introduces additional complexity, including partition management, message ordering considerations, and increased resource requirements.

7. Conclusion and Future Work

This project introduces an end-to-end sentiment analysis pipeline that involves collecting food reviews comments from YouTube, labelling the data, preprocessing the text, building and

evaluating the model, and exporting production ready artefacts. This led to the deployment of two sentiment classification models using multinomial Naive Bayes and LSTM, which were benchmarked and the LSTM model outperformed the multinomial Naive Bayes model with a weighted F1 score of 0.7276 and an accuracy of 73.11% across all three sentiment classes. The pipeline shows how to systematically collect, process and model data from the YouTube comments to derive meaningful sentiments, while all trained artefacts are serialized in a format suitable for Apache Kafka and Apache Spark Structured Streaming for real-time inferences at scale.

Although the findings are encouraging, there are still a few drawbacks. English stopwords and Porter stemming are used in the current pipeline which is not sufficient to capture the multilingual and code switched nature that is present in Malaysian online discourse, where informal colloquial expressions and English are mixed with the Malay language. The boundary between negative and positive classes shows remarkable misclassifications for both models, and the imbalance between the classes in the dataset is also a limitation on the strength of the learned representations, which was mitigated by oversampling.

These limitations should be investigated throughout the pipeline in future studies. In terms of data, the coverage of food channels in Malaysia can be extended to other channels in Malaysia, and the inter-annotator agreement in the labelling can be improved to enhance the quality and diversity of the data. From model construction, the use of transformer-like models such as multilingual and code-switched architecture as natively supported models promises better classification performance. For the streaming layer, a more seamless integration of the Kafka ingestion pipeline with Spark Structured Streaming, with the addition of a model monitoring and retraining mechanism, would guarantee the system's accuracy and scalability over time, as language patterns and food review trends change.

8. References

1. Apache Software Foundation. (n.d.). Apache Kafka. <https://kafka.apache.org/>
2. Apache Software Foundation. (n.d.). Apache Spark. <https://spark.apache.org/>
3. Elastic. (n.d.). Elasticsearch. <https://www.elastic.co/elasticsearch/>
4. Elastic. (n.d.). Kibana. <https://www.elastic.co/kibana/>

9. Appendix

Link to the repository: <https://github.com/drshahizan/HPDP/tree/main/2526/project/p2/TripleA>